

Demonstration:

Code:

src/context/HospitalContext.jsx (Central Hospital State Management, Auth Context & CRUD Actions)

```
import React, { createContext, useContext, useState, useEffect,
useCallback } from 'react';
import {
  INITIAL_USERS, INITIAL_DEPARTMENTS, INITIAL_PATIENTS,
  INITIAL_APPOINTMENTS,
  INITIAL_PRESCRIPTIONS, INITIAL_MEDICINES, INITIAL_LAB_TESTS,
  INITIAL_LAB_REPORTS,
  INITIAL_BILLS, INITIAL_MEDICAL_HISTORY, INITIAL_NOTIFICATIONS,
  INITIAL_SETTINGS,
  INITIAL_ACTIVITY_LOGS
} from '../mockData';
import {
  authService, departmentService, doctorService, patientService,
  appointmentService, prescriptionService, medicineService,
  labService,
  billingService, settingsService
} from '../services/apiServices';

const HospitalContext = createContext(null);

const getStored = (key, fallback) => {
  try {
    const item = localStorage.getItem(`carepoint_${key}`);
    return item ? JSON.parse(item) : fallback;
  } catch (e) {
    return fallback;
  }
};

const ROLE_DEFAULT_EMAILS = {
  Administrator: 'admin@carepoint.com',
  Doctor: 'doctor@carepoint.com',
  Receptionist: 'reception@carepoint.com',
  Pharmacist: 'pharmacy@carepoint.com',
  'Lab Technician': 'lab@carepoint.com',
  Patient: 'patient@carepoint.com'
};

export const HospitalProvider = ({ children }) => {
  const [settings, setSettings] = useState(() => getStored('settings',
  INITIAL_SETTINGS));
  const [users, setUsers] = useState(() => getStored('users',
  INITIAL_USERS));
  const [departments, setDepartments] = useState(() =>
  getStored('departments', INITIAL_DEPARTMENTS));
  const [patients, setPatients] = useState(() => getStored('patients',
  INITIAL_PATIENTS));
  const [appointments, setAppointments] = useState(() =>
  getStored('appointments', INITIAL_APPOINTMENTS));
  const [prescriptions, setPrescriptions] = useState(() =>
  getStored('prescriptions', INITIAL_PRESCRIPTIONS));
  const [medicines, setMedicines] = useState(() =>
  getStored('medicines', INITIAL_MEDICINES));
  const [labTests, setLabTests] = useState(() => getStored('labTests',
  INITIAL_LAB_TESTS));
  const [labReports, setLabReports] = useState(() =>
  getStored('labReports', INITIAL_LAB_REPORTS));
  const [bills, setBills] = useState(() => getStored('bills',
  INITIAL_BILLS));
  const [medicalHistory, setMedicalHistory] = useState(() =>
  getStored('medicalHistory', INITIAL_MEDICAL_HISTORY));
  const [notifications, setNotifications] = useState(() =>
  getStored('notifications', INITIAL_NOTIFICATIONS));
  const [activityLogs, setActivityLogs] = useState(() =>
  getStored('activityLogs', INITIAL_ACTIVITY_LOGS));
  const [isBackendConnected, setIsBackendConnected] = useState(false);

  // Authenticated user state
  const [currentUser, setCurrentUser] = useState(() => {
    const saved = getStored('currentUser', null);
    return saved || INITIAL_USERS[0];
  });

  // Local storage persistence
  useEffect(() => { localStorage.setItem('carepoint_settings',
  JSON.stringify(settings)); }, [settings]);
  useEffect(() => { localStorage.setItem('carepoint_users',
  JSON.stringify(users)); }, [users]);
```

```
    useEffect(() => { localStorage.setItem('carepoint_departments',
  JSON.stringify(departments)); }, [departments]);
    useEffect(() => { localStorage.setItem('carepoint_patients',
  JSON.stringify(patients)); }, [patients]);
    useEffect(() => { localStorage.setItem('carepoint_appointments',
  JSON.stringify(appointments)); }, [appointments]);
    useEffect(() => { localStorage.setItem('carepoint_prescriptions',
  JSON.stringify(prescriptions)); }, [prescriptions]);
    useEffect(() => { localStorage.setItem('carepoint_medicines',
  JSON.stringify(medicines)); }, [medicines]);
    useEffect(() => { localStorage.setItem('carepoint_labTests',
  JSON.stringify(labTests)); }, [labTests]);
    useEffect(() => { localStorage.setItem('carepoint_labReports',
  JSON.stringify(labReports)); }, [labReports]);
    useEffect(() => { localStorage.setItem('carepoint_bills',
  JSON.stringify(bills)); }, [bills]);
    useEffect(() => { localStorage.setItem('carepoint_currentUser',
  JSON.stringify(currentUser)); }, [currentUser]);

    // Synchronize with backend API
    const refreshBackendData = useCallback(async () => {
      try {
        const [deptsRes, patientsRes, apptsRes, rxRes, medsRes,
        testsRes, billsRes] = await Promise.allSettled([
          departmentService.getDepartments(),
          patientService.getPatients(),
          appointmentService.getAppointments(),
          prescriptionService.getPrescriptions(),
          medicineService.getMedicines(),
          labService.getLabTests(),
          billingService.getBills()
        ]);
        if (deptsRes.status === 'fulfilled' &&
        Array.isArray(deptsRes.value)) setDepartments(deptsRes.value);
        if (patientsRes.status === 'fulfilled' &&
        Array.isArray(patientsRes.value)) setPatients(patientsRes.value);
        if (apptsRes.status === 'fulfilled' &&
        Array.isArray(apptsRes.value)) setAppointments(apptsRes.value);
        if (rxRes.status === 'fulfilled' && Array.isArray(rxRes.value))
        setPrescriptions(rxRes.value);
        if (medsRes.status === 'fulfilled' &&
        Array.isArray(medsRes.value)) setMedicines(medsRes.value);
        if (testsRes.status === 'fulfilled' &&
        Array.isArray(testsRes.value)) setLabTests(testsRes.value);
        if (billsRes.status === 'fulfilled' &&
        Array.isArray(billsRes.value)) setBills(billsRes.value);
        setIsBackendConnected(true);
      } catch (err) {
        console.warn('Backend sync in background:', err.message);
      }
    }, []);

    useEffect(() => { refreshBackendData(); }, [refreshBackendData]);

    // Auth Actions
    const login = async (role, customUser = null) => {
      if (customUser) { setCurrentUser(customUser); return customUser; }
      const email = ROLE_DEFAULT_EMAILS[role] ||
      `${role.toLowerCase().replace(/\s+/g, '')}@carepoint.com`;
      try {
        const loginRes = await authService.login(email, 'password123',
        role);
        if (loginRes?.user) {
          const userObj = { ...loginRes.user, name:
          loginRes.user.fullName, avatar: loginRes.user.avatarUrl };
          setCurrentUser(userObj);
          refreshBackendData();
          return userObj;
        }
      } catch (e) {
        console.warn('Local fallback for login:', e.message);
      }
      const user = users.find(u => u.role === role) || {
        id: 'USR-' + Math.floor(100 + Math.random() * 900),
        name: `Demo ${role}`, email, role, status: 'Active'
      };
      setCurrentUser(user);
      return user;
    };

    const logout = async () => {
      try { await authService.logout(); } catch (e) {}
      setCurrentUser(null);
    };
  }
};
```

```

// Patient Registration & Management
const addPatient = async (patientData) => {
  const newId = 'PAT-' + (1000 + patients.length + 1);
  const newPatient = {
    ...patientData, id: newId, patientCode: newId, status: 'Active',
    registeredDate: new Date().toISOString().split('T')[0],
    lastVisit: 'Never'
  };
  setPatients(prev => [newPatient, ...prev]);
  patientService.createPatient(patientData).catch(() => {});
  return newPatient;
};

// Appointment Actions
const addAppointment = async (apptData) => {
  const newId = 'APT-' + (100 + appointments.length + 1);
  const newAppt = { ...apptData, id: newId, appointmentCode: newId,
status: 'Scheduled', createdAt: new Date().toISOString() };
  setAppointments(prev => [newAppt, ...prev]);
  appointmentService.createAppointment(apptData).catch(() => {});
  return newAppt;
};

const updateAppointmentStatus = async (id, status) => {
  setAppointments(prev => prev.map(a => a.id === id ? { ...a, status
} : a));
  appointmentService.updateStatus(id, status).catch(() => {});
};

// Prescription & Pharmacy Dispensation
const addPrescription = async (rxData) => {
  const newId = 'RX-' + (1000 + prescriptions.length + 1);
  const newRx = { ...rxData, id: newId, prescriptionCode: newId,
status: 'Pending', date: new Date().toISOString().split('T')[0] };
  setPrescriptions(prev => [newRx, ...prev]);
  prescriptionService.createPrescription(rxData).catch(() => {});
  return newRx;
};

const dispensePrescription = async (id) => {
  const rx = prescriptions.find(p => p.id === id);
  if (!rx) return;
  setPrescriptions(prev => prev.map(p => p.id === id ? { ...p,
status: 'Dispensed' } : p));
  if (rx.medicinesList) {
    rx.medicinesList.forEach(item => {
      setMedicines(prev => prev.map(m => {
        if (m.name.toLowerCase() === item.name.toLowerCase() ||
med.id === item.id) {
          const updatedStock = Math.max(0, med.stock -
(item.quantity || 1));
          return { ...med, stock: updatedStock, status: updatedStock
<= med.minThreshold ? 'Low Stock' : 'Available' };
        }
      }));
    });
  }
  prescriptionService.dispense(id).catch(() => {});
};

// Billing Actions
const createBill = async (billData) => {
  const newId = 'INV-' + (1000 + bills.length + 1);
  const newBill = { ...billData, id: newId, invoiceNo: newId,
status: 'Unpaid', date: new Date().toISOString().split('T')[0] };
  setBills(prev => [newBill, ...prev]);
  billingService.createBill(billData).catch(() => {});
  return newBill;
};

const markBillPaid = async (id, paymentMethod = 'Credit Card') => {
  setBills(prev => prev.map(b => b.id === id ? { ...b, status:
'Paid', paymentMethod, paidDate: new Date().toISOString().split('T')
[0] } : b));
  billingService.updateStatus(id, 'Paid').catch(() => {});
};

return (
  <HospitalContext.Provider value={{
    currentUser, settings, users, departments, patients,
    appointments,
    prescriptions, medicines, labTests, labReports, bills,
    medicalHistory,
    notifications, activityLogs, isBackendConnected, login, logout,

```

```

    addPatient, addAppointment, updateAppointmentStatus,
    addPrescription,
    dispensePrescription, createBill, markBillPaid,
    refreshBackendData
  }}>
  {children}
</HospitalContext.Provider>
);
};

```

```

export const useHospital = () => {
  const context = useContext(HospitalContext);
  if (!context) throw new Error('useHospital must be used within
HospitalProvider');
  return context;
};

```

src/services/apiServices.js (Centralized REST API Client & Interceptors Layer)

```

import axios from 'axios';

const API_BASE_URL = import.meta.env.VITE_API_URL ||
'http://localhost:5000/api';

export const apiClient = axios.create({
  baseURL: API_BASE_URL,
  timeout: 15000,
  headers: { 'Content-Type': 'application/json' }
});

// Attach Supabase/JWT authentication token & user headers
apiClient.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('carepoint_token');
    const userRole = localStorage.getItem('carepoint_user_role');
    const userEmail = localStorage.getItem('carepoint_user_email');
    if (token) config.headers.Authorization = `Bearer ${token}`;
    if (userRole) config.headers['x-user-role'] = userRole;
    if (userEmail) config.headers['x-user-email'] = userEmail;
    return config;
  },
  (error) => Promise.reject(error)
);

// Response interceptor for unwrapping standardized API envelope
apiClient.interceptors.response.use(
  (response) => {
    if (response.data && response.data.data !== undefined) return
response.data.data;
    return response.data;
  },
  (error) => {
    const message = error.response?.data?.message || error.message ||
'API request failed';
    console.warn(`[HMS API Error] ${error.config?.url}:`, message);
    return Promise.reject(error);
  }
);

// 1. Authentication Services
export const authService = {
  login: async (email, password, role) => {
    const data = await apiClient.post('/auth/login', { email,
password, role });
    if (data?.token || data?.session?.accessToken) {
      localStorage.setItem('carepoint_token', data.token ||
data.session.accessToken);
      if (data.user?.role) localStorage.setItem('carepoint_user_role',
data.user.role);
      if (data.user?.email)
        localStorage.setItem('carepoint_user_email', data.user.email);
    }
    return data;
  },
  getMe: async () => apiClient.get('/auth/me'),
  getProfile: async () => apiClient.get('/auth/profile'),
  registerPatient: async (patientData) =>
    apiClient.post('/auth/register', patientData),
  logout: async () => {
    try { await apiClient.post('/auth/logout'); } finally {
      localStorage.removeItem('carepoint_token');
      localStorage.removeItem('carepoint_user_role');
      localStorage.removeItem('carepoint_user_email');
    }
  }
};

```

```

});

// 2. Department Services
export const departmentService = {
  getDepartments: async () => apiClient.get('/departments'),
  getById: async (id) => apiClient.get(`/departments/${id}`),
  createDepartment: async (data) => apiClient.post('/departments',
data)
};

// 3. Patient Services
export const patientService = {
  getPatients: async () => apiClient.get('/patients'),
  getById: async (id) => apiClient.get(`/patients/${id}`),
  createPatient: async (data) => apiClient.post('/patients', data),
  updatePatient: async (id, data) => apiClient.put(`/patients/${id}`,
data)
};

// 4. Appointment Services
export const appointmentService = {
  getAppointments: async (filter = {}) =>
apiClient.get('/appointments', { params: filter }),
  getById: async (id) => apiClient.get(`/appointments/${id}`),
  createAppointment: async (data) => apiClient.post('/appointments',
data),
  updateStatus: async (id, status) =>
apiClient.patch(`/appointments/${id}/status`, { status }),
  reschedule: async (id, date, time) =>
apiClient.patch(`/appointments/${id}/reschedule`, { date, time }),
  cancel: async (id, reason) =>
apiClient.post(`/appointments/${id}/cancel`, { reason })
};

// 5. Prescription Services
export const prescriptionService = {
  getPrescriptions: async () => apiClient.get('/prescriptions'),
  createPrescription: async (data) => apiClient.post('/prescriptions',
data),
  dispense: async (id) =>
apiClient.post(`/prescriptions/${id}/dispense`)
};

// 6. Medicine & Pharmacy Inventory
export const medicineService = {
  getMedicines: async () => apiClient.get('/medicines'),
  createMedicine: async (data) => apiClient.post('/medicines', data),
  updateStock: async (id, stock) =>
apiClient.patch(`/medicines/${id}/stock`, { stock })
};

// 7. Lab Diagnostic Services
export const labService = {
  getLabTests: async () => apiClient.get('/lab/tests'),
  requestTest: async (data) => apiClient.post('/lab/tests', data),
  getLabReports: async () => apiClient.get('/lab/reports')
};

// 8. Billing Services
export const billingService = {
  getBills: async () => apiClient.get('/billing'),
  createBill: async (data) => apiClient.post('/billing', data),
  updateStatus: async (id, status) =>
apiClient.patch(`/billing/${id}/status`, { status })
};

```

src/App.jsx (Application Routing, Multi-Role Portal Navigation & Layout Shell)

```

import React from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from
'react-router-dom';
import { HospitalProvider } from './context/HospitalContext';
import { PublicLandingPage } from './pages/PublicPages';
import { LoginPage, RegisterPage, ForgotPasswordPage,
ResetPasswordPage } from './pages/AuthPages';
import { PatientPortal } from './pages/PatientPortal';
import { DoctorPortal } from './pages/DoctorPortal';
import { ReceptionistPortal } from './pages/ReceptionistPortal';
import { PharmacistPortal } from './pages/PharmacistPortal';
import { LabTechnicianPortal } from './pages/LabTechnicianPortal';
import { AdminPortal } from './pages/AdminPortal';

export function App() {
  return (
    <HospitalProvider>

```

```

<Router>
  <Routes>
    { /* Public Landing & Marketing */ }
    <Route path="/" element={<PublicLandingPage />} />
    <Route path="/departments" element={<PublicLandingPage />} />

    <Route path="/doctors" element={<PublicLandingPage />} />
    <Route path="/services" element={<PublicLandingPage />} />
    <Route path="/contact" element={<PublicLandingPage />} />

    { /* Authentication Pages */ }
    <Route path="/login" element={<LoginPage />} />
    <Route path="/register" element={<RegisterPage />} />
    <Route path="/forgot-password" element={<ForgotPasswordPage
/>} />
    <Route path="/reset-password" element={<ResetPasswordPage
/>} />

    { /* Patient Portal Routes */ }
    <Route path="/patient" element={<PatientPortal
view="dashboard" />} />
    <Route path="/patient/book" element={<PatientPortal
view="book" />} />
    <Route path="/patient/appointments" element={<PatientPortal
view="appointments" />} />
    <Route path="/patient/history" element={<PatientPortal
view="history" />} />
    <Route path="/patient/prescriptions" element={<PatientPortal
view="prescriptions" />} />
    <Route path="/patient/lab-reports" element={<PatientPortal
view="lab-reports" />} />
    <Route path="/patient/bills" element={<PatientPortal
view="bills" />} />

    { /* Doctor Portal Routes */ }
    <Route path="/doctor" element={<DoctorPortal
view="dashboard" />} />
    <Route path="/doctor/appointments" element={<DoctorPortal
view="appointments" />} />
    <Route path="/doctor/patients" element={<DoctorPortal
view="patients" />} />
    <Route path="/doctor/records" element={<DoctorPortal
view="records" />} />
    <Route path="/doctor/prescriptions" element={<DoctorPortal
view="prescriptions" />} />
    <Route path="/doctor/lab-tests" element={<DoctorPortal
view="lab-tests" />} />

    { /* Receptionist Portal Routes */ }
    <Route path="/receptionist" element={<ReceptionistPortal
view="dashboard" />} />
    <Route path="/receptionist/register" element=
{<ReceptionistPortal view="register" />} />
    <Route path="/receptionist/appointments" element=
{<ReceptionistPortal view="appointments" />} />
    <Route path="/receptionist/billing" element=
{<ReceptionistPortal view="billing" />} />

    { /* Pharmacist Portal Routes */ }
    <Route path="/pharmacist" element={<PharmacistPortal
view="dashboard" />} />
    <Route path="/pharmacist/prescriptions" element=
{<PharmacistPortal view="prescriptions" />} />
    <Route path="/pharmacist/inventory" element=
{<PharmacistPortal view="inventory" />} />
    <Route path="/pharmacist/dispense" element=
{<PharmacistPortal view="dispense" />} />

    { /* Lab Technician & Admin Portals */ }
    <Route path="/lab" element={<LabTechnicianPortal
view="dashboard" />} />
    <Route path="/lab/tests" element={<LabTechnicianPortal
view="tests" />} />
    <Route path="/lab/reports" element={<LabTechnicianPortal
view="reports" />} />
    <Route path="/admin" element={<AdminPortal view="dashboard"
/>} />
    <Route path="/admin/users" element={<AdminPortal
view="users" />} />
    <Route path="/admin/departments" element={<AdminPortal
view="departments" />} />
    <Route path="/admin/billing" element={<AdminPortal
view="billing" />} />

    { /* Fallback */ }

```

```

        <Route path="*" element={<Navigate to="/" replace />} />
      </Routes>
    </Router>
  </HospitalProvider>
);
}

export default App;

src/pages/DoctorPortal.jsx (Doctor Clinical Workspace, OPD
Consultations & E-Prescriptions)

import React, { useState } from 'react';
import { useHospital } from '../context/HospitalContext';
import { DashboardLayout, PageHeader, DataTable, StatusBadge, Modal }
from '../components/CommonComponents';
import { Calendar, Users, FileText, Pill, FlaskConical, Stethoscope,
Clock, CheckCircle2, Plus } from 'lucide-react';

export const DoctorPortal = ({ view = 'dashboard' }) => {
  const {
    currentUser, patients, appointments, updateAppointmentStatus,
    prescriptions, addPrescription, labTests, requestLabTest,
    medicines
  } = useHospital();

  const currentDocName = currentUser?.name || 'Dr. Sarah Jenkins';
  const docAppointments = appointments.filter(a =>
a.doctorName?.toLowerCase().includes(currentDocName.toLowerCase()) ||
a.doctorId === currentUser?.id);
  const [activeModal, setActiveModal] = useState(null);
  const [selectedRecord, setSelectedRecord] = useState(null);

  // Consultation & Vitals Form State
  const [consultForm, setConsultForm] = useState({
    patientId: 'PAT-1001', patientName: 'Johnathan Doe',
    appointmentId: '',
    diagnosis: '', notes: '', bp: '120/80 mmHg', pulse: '72 bpm',
    weight: '70 kg',
    temperature: '98.6 °F', prescribeNow: false, orderLabNow: false
  });

  // E-Prescription State
  const [rxForm, setRxForm] = useState({
    patientId: 'PAT-1001', patientName: 'Johnathan Doe', diagnosis:
'',
    instructions: 'Take medications as prescribed after meals. Hydrate
well.',
    medicinesList: [
      { id: 'MED-101', name: 'Amlodipine 5mg', dosage: '5mg',
frequency: 'Once daily (Morning)', duration: '30 Days', quantity: 30 }
    ]
  });

  const handleStartConsultation = (appt) => {
    setSelectedRecord(appt);
    setConsultForm({
      patientId: appt.patientId, patientName: appt.patientName,
      appointmentId: appt.id,
      diagnosis: '', notes: '', bp: '120/80 mmHg', pulse: '74 bpm',
      weight: '72 kg',
      temperature: '98.4 °F', prescribeNow: false, orderLabNow: false
    });
    setActiveModal('consultation');
  };

  const handleSaveConsultation = (e) => {
    e.preventDefault();
    if (selectedRecord) {
      updateAppointmentStatus(selectedRecord.id, 'Completed');
    }
    if (consultForm.prescribeNow) {
      addPrescription({
        patientId: consultForm.patientId, patientName:
consultForm.patientName,
        doctorId: currentUser?.id || 'USR-002', doctorName:
currentDocName,
        diagnosis: consultForm.diagnosis, instructions:
rxForm.instructions,
        medicinesList: rxForm.medicinesList
      });
      setActiveModal(null);
    }
  };

  const handleAddMedicineRow = () => {

```

```

    setRxForm({
      ...rxForm,
      medicinesList: [
        ...rxForm.medicinesList,
        { id: 'MED-' + Date.now().toString().slice(-3), name:
medicines[0]?.name || 'Amoxicillin 500mg', dosage: '500mg', frequency:
'Twice daily', duration: '5 Days', quantity: 10 }
      ]
    });
  };

  return (
    <DashboardLayout title="Doctor Clinical Workstation">
      <PageHeader
        title={`Dr. Workspace - ${currentDocName}`}
        subtitle="Manage daily OPD appointments, record clinical
consultations, and issue e-prescriptions"
      />

      <div className="grid grid-cols-1 lg:grid-cols-3 gap-6 mb-8">
        <div className="bg-white p-5 rounded-xl border border-slate-
200 shadow-xs">
          <div className="flex items-center gap-3">
            <div className="p-3 bg-blue-50 text-blue-600 rounded-lg">
              <Calendar className="w-6 h-6" /></div>
            <div>
              <p className="text-xs font-semibold text-slate-500
uppercase">Today's Schedule</p>
              <h3 className="text-2xl font-bold text-slate-900">
                {docAppointments.length} Patients</h3>
            </div>
          </div>
          <div className="bg-white p-5 rounded-xl border border-slate-
200 shadow-xs">
            <div className="flex items-center gap-3">
              <div className="p-3 bg-emerald-50 text-emerald-600
rounded-lg"><CheckCircle2 className="w-6 h-6" /></div>
              <div>
                <p className="text-xs font-semibold text-slate-500
uppercase">Completed Consults</p>
                <h3 className="text-2xl font-bold text-slate-900">
                  {docAppointments.filter(a => a.status === 'Completed').length}
                  Consults</h3>
              </div>
            </div>
          </div>
          <div className="bg-white p-5 rounded-xl border border-slate-
200 shadow-xs">
            <div className="flex items-center gap-3">
              <div className="p-3 bg-purple-50 text-purple-600 rounded-
lg"><Pill className="w-6 h-6" /></div>
              <div>
                <p className="text-xs font-semibold text-slate-500
uppercase">Prescriptions Issued</p>
                <h3 className="text-2xl font-bold text-slate-900">
                  {prescriptions.length} Records</h3>
              </div>
            </div>
          </div>
        </div>

        {/* OPD Appointments Queue */}
        <div className="bg-white rounded-xl border border-slate-200
shadow-xs overflow-hidden">
          <div className="p-4 border-b border-slate-100 flex justify-
between items-center">
            <h2 className="text-sm font-bold text-slate-900">OPD Patient
Queue & Consultations</h2>
          </div>
          <DataTable
            columns={[
              { header: 'Patient', accessor: 'patientName' },
              { header: 'Time Slot', accessor: 'time' },
              { header: 'Reason', accessor: 'reason' },
              { header: 'Status', accessor: (row) => <StatusBadge
status={row.status} /> },
              {
                header: 'Actions',
                accessor: (row) => (
                  <button
                    onClick={() => handleStartConsultation(row)}
                    className="px-3 py-1.5 bg-rose-600 hover:bg-rose-700
text-white rounded-lg text-xs font-semibold"

```

```

        Consult Patient
      </button>
    )
  }
}
data={docAppointments}
/>
</div>

{/* Consultation & Vitals Modal */}
{activeModal === 'consultation' && (
  <Modal title={`Clinical Consultation -
${consultForm.patientName}`} onClose={() => setActiveModal(null)}>
    <form onSubmit={handleSaveConsultation} className="space-y-
4">
      <div className="grid grid-cols-2 sm:grid-cols-4 gap-3 bg-
slate-50 p-3 rounded-lg border border-slate-200">
        <div><label className="text-[11px] font-semibold text-
slate-600">BP</label><input value={consultForm.bp} onChange={e =>
setConsultForm({...consultForm, bp: e.target.value}} className="w-
full text-xs p-1.5 border rounded" /></div>
        <div><label className="text-[11px] font-semibold text-
slate-600">Pulse</label><input value={consultForm.pulse} onChange={e
=> setConsultForm({...consultForm, pulse: e.target.value}}>
className="w-full text-xs p-1.5 border rounded" /></div>
        <div><label className="text-[11px] font-semibold text-
slate-600">Weight</label><input value={consultForm.weight} onChange={e
=> setConsultForm({...consultForm, weight: e.target.value}}>
className="w-full text-xs p-1.5 border rounded" /></div>
        <div><label className="text-[11px] font-semibold text-
slate-600">Temp</label><input value={consultForm.temperature}
onChange={e => setConsultForm({...consultForm, temperature:
e.target.value}} className="w-full text-xs p-1.5 border rounded" />
</div>
      </div>
      <div>
        <label className="text-xs font-semibold text-slate-
700">Clinical Diagnosis</label>
        <input required value={consultForm.diagnosis} onChange=
{e => setConsultForm({...consultForm, diagnosis: e.target.value}}>
placeholder="e.g. Essential Hypertension Grade I" className="w-full
text-xs p-2 border rounded-lg mt-1" />
      </div>
      <div>
        <label className="text-xs font-semibold text-slate-
700">Clinical Examination & Doctor Notes</label>
        <textarea value={consultForm.notes} onChange={e =>
setConsultForm({...consultForm, notes: e.target.value}} rows={3}>
placeholder="Observations, symptoms, and dietary recommendations..."
className="w-full text-xs p-2 border rounded-lg mt-1" />
      </div>
      <div className="flex items-center gap-2">
        <input type="checkbox" id="rxCheck" checked=
{consultForm.prescribeNow} onChange={e =>
setConsultForm({...consultForm, prescribeNow: e.target.checked}} />
        <label htmlFor="rxCheck" className="text-xs font-
semibold text-slate-700">Issue Electronic Prescription (E-Rx)</label>
      </div>
      {consultForm.prescribeNow && (
        <div className="p-3 bg-slate-50 border rounded-lg space-
y-2">
          <div className="flex justify-between items-center">
            <span className="text-xs font-bold text-slate-
800">Prescription Medications</span>
            <button type="button" onClick={handleAddMedicineRow}
className="text-xs text-rose-600 font-semibold">+ Add Drug</button>
          </div>
          {rxForm.medicinesList.map((m, idx) => (
            <div key={idx} className="grid grid-cols-4 gap-2">
              <input value={m.name} onChange={e => { const list
= [...rxForm.medicinesList]; list[idx].name = e.target.value;
setRxForm({...rxForm, medicinesList: list}); }} className="text-xs p-
1.5 border rounded" />
              <input value={m.dosage} onChange={e => { const
list = [...rxForm.medicinesList]; list[idx].dosage = e.target.value;
setRxForm({...rxForm, medicinesList: list}); }} className="text-xs p-
1.5 border rounded" />
              <input value={m.frequency} onChange={e => { const
list = [...rxForm.medicinesList]; list[idx].frequency =
e.target.value; setRxForm({...rxForm, medicinesList: list}); }}
className="text-xs p-1.5 border rounded" />
              <input value={m.duration} onChange={e => { const
list = [...rxForm.medicinesList]; list[idx].duration = e.target.value;
setRxForm({...rxForm, medicinesList: list}); }} className="text-xs p-
1.5 border rounded" />
            </div>
          )
        )
      )
    )
  )
}

```

```

    </div>
  )
}
</div>
)
<div className="flex justify-end gap-2 pt-2">
  <button type="button" onClick={() =>
setActiveModal(null)} className="px-4 py-2 border rounded-lg text-
xs">Cancel</button>
  <button type="submit" className="px-4 py-2 bg-rose-600
text-white rounded-lg text-xs font-semibold">Complete
Consultation</button>
</div>
</form>
</Modal>
))
</DashboardLayout>
);
};

```

src/pages/PatientPortal.jsx (Patient Self-Service Dashboard, Appointments & Lab Results)

```

import React, { useState } from 'react';
import { useHospital } from '../context/HospitalContext';
import { DashboardLayout, PageHeader, DataTable, StatusBadge, Modal }
from '../components/CommonComponents';
import { Calendar, CalendarPlus, Pill, FlaskConical, Receipt, Clock,
CheckCircle2, ArrowRight } from 'lucide-react';

export const PatientPortal = ({ view = 'dashboard' }) => {
  const {
    currentUser, patients, appointments, addAppointment,
    prescriptions, labReports, bills, markBillPaid, departments, users
  } = useHospital();

  const currentPatId = currentUser?.patientId || 'PAT-1001';
  const patientData = patients.find(p => p.id === currentPatId) ||
patients[0] || {};
  const myAppointments = appointments.filter(a => a.patientId ===
currentPatId || a.patientName === patientData.name);
  const myPrescriptions = prescriptions.filter(p => p.patientId ===
currentPatId || p.patientName === patientData.name);
  const myBills = bills.filter(b => b.patientId === currentPatId ||
b.patientName === patientData.name);

  const [bookingForm, setBookingForm] = useState({
    department: 'Cardiology', doctorId: 'USR-002', doctorName: 'Dr.
Sarah Jenkins',
    date: new Date(Date.now() + 86400000).toISOString().split('T')[0],
    time: '10:00 AM', reason: '', type: 'Regular Consultation'
  });
  const [bookingSuccess, setBookingSuccess] = useState(false);
  const [activeModal, setActiveModal] = useState(null);
  const [selectedBill, setSelectedBill] = useState(null);

  const handleBookSubmit = (e) => {
    e.preventDefault();
    addAppointment({
      ...bookingForm, patientId: currentPatId, patientName:
patientData.name || patientData.fullName
    });
    setBookingSuccess(true);
    setTimeout(() => { setBookingSuccess(false); }, 3000);
  };

  const handlePayBill = (bill) => {
    setSelectedBill(bill);
    setActiveModal('payBill');
  };

  const confirmBillPayment = () => {
    if (selectedBill) {
      markBillPaid(selectedBill.id, 'Credit Card');
      setActiveModal(null);
    }
  };

  return (
    <DashboardLayout title="Patient Self-Service Portal">
      <PageHeader
        title={`Welcome, ${patientData.name || currentUser?.name}`}
        subtitle={`Patient ID: ${currentPatId} | Blood Group:
${patientData.bloodGroup || 'O+'} | Status: Active`}
      />
      <div className="grid grid-cols-1 md:grid-cols-4 gap-4 mb-6">

```

```

    <div className="bg-white p-4 rounded-xl border border-slate-200">
      <p className="text-xs text-slate-500 font-semibold">Upcoming Visits</p>
      <h4 className="text-xl font-bold text-slate-900">
        {myAppointments.filter(a => a.status === 'Scheduled').length}</h4>
      </div>
      <div className="bg-white p-4 rounded-xl border border-slate-200">
        <p className="text-xs text-slate-500 font-semibold">Active Prescriptions</p>
        <h4 className="text-xl font-bold text-slate-900">
          {myPrescriptions.length}</h4>
        </div>
        <div className="bg-white p-4 rounded-xl border border-slate-200">
          <p className="text-xs text-slate-500 font-semibold">Diagnostic Reports</p>
          <h4 className="text-xl font-bold text-slate-900">
            {labReports.length}</h4>
          </div>
          <div className="bg-white p-4 rounded-xl border border-slate-200">
            <p className="text-xs text-slate-500 font-semibold">Pending Invoices</p>
            <h4 className="text-xl font-bold text-rose-600">
              {myBills.filter(b => b.status === 'Unpaid').length}</h4>
            </div>
            </div>

            { /* Online Appointment Booking Wizard */ }
            <div className="bg-white p-6 rounded-xl border border-slate-200 shadow-xs mb-8">
              <h3 className="text-base font-bold text-slate-900 mb-4 flex items-center gap-2">
                <CalendarPlus className="w-5 h-5 text-rose-600" /> Book Clinical Consultation
              </h3>
              {bookingSuccess && (
                <div className="mb-4 p-3 bg-emerald-50 border border-emerald-200 text-emerald-800 rounded-lg text-xs font-semibold">
                  Appointment successfully scheduled! Check your upcoming visits list below.
                </div>
              )}
              <form onSubmit={handleBookSubmit} className="grid grid-cols-1 md:grid-cols-3 gap-4">
                <div>
                  <label className="block text-xs font-semibold text-slate-700 mb-1">Clinical Department</label>
                  <select
                    value={bookingForm.department}
                    onChange={(e) => {
                      const dept = e.target.value;
                      const doc = users.find(u => u.role === 'Doctor' && u.department === dept);
                      setBookingForm({ ...bookingForm, department: dept, doctorName: doc ? doc.name : 'Dr. Sarah Jenkins' });
                    }}
                    className="w-full text-xs p-2.5 border rounded-lg bg-white">
                    <div>
                      {departments.map(d => <option key={d.id} value={d.name}>{d.name}</option>)}
                    </div>
                  </div>
                  <div>
                    <label className="block text-xs font-semibold text-slate-700 mb-1">Preferred Date</label>
                    <input
                      type="date"
                      required
                      value={bookingForm.date}
                      onChange={(e) => setBookingForm({ ...bookingForm, date: e.target.value })}
                      className="w-full text-xs p-2.5 border rounded-lg"
                    />
                  </div>
                  <div>
                    <label className="block text-xs font-semibold text-slate-700 mb-1">Time Slot</label>
                    <select
                      value={bookingForm.time}
                      onChange={(e) => setBookingForm({ ...bookingForm, time: e.target.value })}

```

```

    className="w-full text-xs p-2.5 border rounded-lg bg-white">
      <div>
        <option value="09:00 AM">09:00 AM</option>
        <option value="10:00 AM">10:00 AM</option>
        <option value="11:30 AM">11:30 AM</option>
        <option value="02:00 PM">02:00 PM</option>
        <option value="04:00 PM">04:00 PM</option>
      </select>
      </div>
      <div className="md:col-span-2">
        <label className="block text-xs font-semibold text-slate-700 mb-1">Reason for Visit / Symptoms</label>
        <input
          required
          placeholder="e.g. Routine cardiovascular checkup, chest discomfort..."
          value={bookingForm.reason}
          onChange={(e) => setBookingForm({ ...bookingForm, reason: e.target.value })}
          className="w-full text-xs p-2.5 border rounded-lg"
        />
      </div>
      <div className="flex items-end">
        <button type="submit" className="w-full py-2.5 bg-rose-600 hover:bg-rose-700 text-white rounded-lg text-xs font-semibold flex items-center justify-center gap-2">
          Confirm & Book Slot <ArrowRight className="w-4 h-4" />
        </button>
      </div>
    </form>
  </div>

  { /* Patient Invoices & Checkout */ }
  <div className="bg-white rounded-xl border border-slate-200 overflow-hidden mb-8">
    <div className="p-4 border-b border-slate-100"><h3
      className="text-sm font-bold text-slate-900">Hospital Billing & Invoices</h3></div>
    <DataTable
      columns={[
        { header: 'Invoice #', accessor: 'invoiceNo' },
        { header: 'Date', accessor: 'date' },
        { header: 'Amount ($)', accessor: (row) => `$$${row.totalAmount} || row.amount` },
        { header: 'Status', accessor: (row) => <StatusBadge status={row.status} /> },
        {
          header: 'Action',
          accessor: (row) => row.status === 'Unpaid' ? (
            <button onClick={() => handlePayBill(row)}
              className="px-3 py-1 bg-emerald-600 text-white rounded-lg text-xs font-semibold">
                Pay Now
            </button>
          ) : <span className="text-xs text-slate-400 font-semibold">Settled</span>
        }
      ]}
      data={myBills}
    />
  </div>

  { /* Pay Bill Modal */ }
  {activeModal === 'payBill' && (
    <Modal title={`Pay Invoice #${selectedBill?.invoiceNo}`}
      onClose={() => setActiveModal(null)}>
      <div className="space-y-4">
        <div className="p-4 bg-slate-50 border rounded-lg text-center">
          <p className="text-xs text-slate-500">Total Outstanding Balance</p>
          <h2 className="text-2xl font-bold text-slate-900 mt-1">${selectedBill?.totalAmount} || selectedBill?.amount</h2>
        </div>
        <div>
          <label className="text-xs font-semibold text-slate-700">Payment Gateway</label>
          <select className="w-full text-xs p-2 border rounded-lg mt-1 bg-white">
            <option>Credit / Debit Card (Instant Gateway)</option>
            <option>Health Insurance Direct Claim</option>
            <option>Hospital Digital Wallet</option>
          </select>
        </div>
      </div>
    </Modal>
  )}

```



```
import React, { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useHospital } from '../context/HospitalContext';
import { HeartPulse, Mail, Lock, UserCheck, AlertCircle } from 'lucide-react';
```

```

export const LoginPage = () => {
  const { login, users } = useHospital();
  const navigate = useNavigate();

  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');

  const handleLoginSubmit = (e) => {
    e.preventDefault();
    if (!email || !password) {
      setError('Please enter email and password.');
```

```

slate-700 mb-1">Email Address</label>
  <input
    type="email"
    required
    value={email}
    onChange={(e) => setEmail(e.target.value)}
    placeholder="doctor@carepoint.com"
    className="w-full text-xs p-2.5 border rounded-lg"
  />
</div>
<div>
  <label className="block text-xs font-semibold text-
slate-700 mb-1">Password</label>
  <input
    type="password"
    required
    value={password}
    onChange={(e) => setPassword(e.target.value)}
    placeholder="....."
    className="w-full text-xs p-2.5 border rounded-lg"
  />
</div>
<button type="submit" className="w-full py-2.5 bg-rose-600
text-white rounded-lg text-xs font-semibold">
  Sign In to Workstation
</button>
</form>

{/* Quick Demo Role Selector */}
<div className="mt-6 pt-6 border-t border-slate-100">
  <p className="text-[11px] font-bold text-slate-400
uppercase tracking-wider text-center mb-3">
    One-Click Role Demo Sign In
  </p>
  <div className="grid grid-cols-2 gap-2">
    {[ 'Doctor', 'Patient', 'Receptionist', 'Pharmacist',
'Lab Technician', 'Administrator'].map((role) => (
      <button
        key={role}
        type="button"
        onClick={() => handleQuickRoleSelect(role)}
        className="p-2 bg-slate-50 hover:bg-rose-50 border
border-slate-200 hover:border-rose-300 rounded-lg text-xs font-
semibold text-slate-700 hover:text-rose-700 text-left flex items-
center gap-1.5"
      >
        <UserCheck className="w-3.5 h-3.5" /> {role}
      </button>
    ))}
  </div>
</div>
</div>
</div>
);
};

```

```
import express from 'express';
import cors from 'cors';
import morgan from 'morgan';
import config from './config/config.js';
import apiRoutes from './routes/index.js';
import { notFoundHandler, errorHandler } from
'./middleware/errorMiddleware.js';
```

```
const app = express();

// CORS configuration for multi-origin client requests
app.use(cors({
  origin: (origin, callback) => {
    if (!origin || origin.includes('localhost') ||
origin.includes('127.0.0.1') || origin === config.frontendUrl) {
      callback(null, true);
    } else {
      callback(null, true);
    }
  },
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization', 'x-user-role', 'x-user-email']
})));
```

```

if (config.nodeEnv !== 'test') {
  app.use(morgan('dev'));
}

// Request Parsers
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true, limit: '10mb' }));

```

```

// Health & Root Status Route
app.get('/', (req, res) => {
  res.json({
    name: 'CarePoint Hospital Management System API',
    version: '1.0.0',
    status: 'online',
    documentation: '/api/health'
  });
});

```

```

// Mount Main API Router Gateway
app.use('/api', apiRoutes);

```

```

// Catch-all 404 & Centralized Error Handlers
app.use(notFoundHandler);
app.use(errorHandler);

```

```

export default app;

```

backend/src/middleware/authMiddleware.js (JWT Authentication & Role-Based Access Guard)

```

import { supabasePublic } from '../lib/supabase.js';
import db from '../lib/db.js';
import { ErrorResponse } from '../utils/response.js';

/**
 * Authentication Middleware: Extracts JWT Bearer token, validates via
 * Supabase Auth,
 * attaches HMS user profile and resolved doctorId/patientId to
 * req.user.
 */
export const authenticate = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return ErrorResponse(res, 401, 'Authentication token required. Please sign in.');
```

```

export const appointmentController = {
  getAll: async (req, res, next) => {
    try {
      const appointments = await appointmentService.getAll(req.query);
      return sendSuccess(res, 200, 'Appointments retrieved successfully', appointments);
    } catch (err) { next(err); }
  },

  getById: async (req, res, next) => {
    try {
      const appointment = await appointmentService.getById(req.params.id);
      if (!appointment) return sendNotFound(res, 'Appointment not found');
      return sendSuccess(res, 200, 'Appointment retrieved', appointment);
    } catch (err) { next(err); }
  },

  create: async (req, res, next) => {
    try {
      const appointment = await appointmentService.create(req.body, req.user);
      return sendSuccess(res, 201, 'Appointment booked successfully', appointment);
    } catch (err) { next(err); }
  },

  updateStatus: async (req, res, next) => {
    try {
      const { status } = req.body;
      const updated = await appointmentService.updateStatus(req.params.id, status, req.user);
      if (!updated) return sendNotFound(res, 'Appointment not found');
      return sendSuccess(res, 200, `Appointment marked as ${status}`, updated);
    } catch (err) { next(err); }
  },

  reschedule: async (req, res, next) => {
    try {
      const { date, time } = req.body;
      const updated = await appointmentService.reschedule(req.params.id, date, time, req.user);
      if (!updated) return sendNotFound(res, 'Appointment not found');
      return sendSuccess(res, 200, 'Appointment rescheduled successfully', updated);
    } catch (err) { next(err); }
  },

  cancel: async (req, res, next) => {
    try {
      const { reason } = req.body;
      const cancelled = await appointmentService.cancel(req.params.id, reason, req.user);
      if (!cancelled) return sendNotFound(res, 'Appointment not found');
      return sendSuccess(res, 200, 'Appointment cancelled successfully', cancelled);
    } catch (err) { next(err); }
  }
};

```

backend/src/services/appointmentService.js (Appointment Business Logic, Conflict Detection & Notifications)

```

import db from '../lib/db.js';

export const appointmentService = {
  getAll: async (filter = {}) => {
    let list = await db.getAppointments();
    if (filter.patientId) list = list.filter(a => a.patientId === filter.patientId || a.patientName === filter.patientName);
    if (filter.doctorId) list = list.filter(a => a.doctorId === filter.doctorId || a.doctorName === filter.doctorName);
    if (filter.status) list = list.filter(a => a.status === filter.status);
    if (filter.date) list = list.filter(a => a.date === filter.date);
    return list;
  },

  getById: async (id) => db.findAppointmentById(id),

  create: async (data, currentUser) => {
    // Conflict detection: prevent booking if doctor is already booked

```

```

    for this date and time
    const existing = await db.getAppointments();
    const hasConflict = existing.some(
      a => (a.doctorId === data.doctorId || a.doctorName === data.doctorName) &&
        a.date === data.date &&
        a.time === data.time &&
        a.status === 'Scheduled'
    );

    if (hasConflict) {
      throw new Error(`Doctor is already booked for ${data.date} at ${data.time}. Please select another time slot.`);
    }

    const appt = await db.createAppointment({
      ...data,
      patientId: data.patientId || currentUser?.patientId,
      patientName: data.patientName || currentUser?.fullName
    });

    // Notify doctor and receptionist staff
    await db.createNotification({
      targetRole: 'Doctor',
      title: 'New Appointment Scheduled',
      message: `${appt.patientName} booked consultation for ${appt.date} at ${appt.time}.`
    });

    await db.createNotification({
      targetRole: 'Receptionist',
      title: 'New OPD Booking',
      message: `Appointment ${appt.appointmentCode} registered for ${appt.patientName}.`
    });

    return appt;
  },

  updateStatus: async (id, status, currentUser) => {
    const updated = await db.updateAppointment(id, { status });
    if (updated) {
      await db.createNotification({
        profileId: updated.patientId,
        targetRole: 'Patient',
        title: `Appointment ${status}`,
        message: `Your appointment with ${updated.doctorName} is now marked as ${status}.`
      });
    }
    return updated;
  },

  reschedule: async (id, date, time, currentUser) => {
    return await db.updateAppointment(id, { date, time, status: 'Scheduled' });
  },

  cancel: async (id, reason, currentUser) => {
    return await db.updateAppointment(id, { status: 'Cancelled', cancelReason: reason });
  }
};

```

backend/src/controllers/prescriptionController.js (Prescription Creation & Pharmacy Dispensing Lifecycle)

```

import { prescriptionService } from '../services/prescriptionService.js';
import { sendSuccess, sendNotFound } from '../utils/response.js';

export const prescriptionController = {
  getAll: async (req, res, next) => {
    try {
      const prescriptions = await prescriptionService.getAll(req.query);
      return sendSuccess(res, 200, 'Prescriptions retrieved', prescriptions);
    } catch (err) { next(err); }
  },

  getById: async (req, res, next) => {
    try {
      const prescription = await prescriptionService.getById(req.params.id);
      if (!prescription) return sendNotFound(res, 'Prescription not

```

```

found');
    return sendSuccess(res, 200, 'Prescription retrieved',
prescription);
  } catch (err) { next(err); }
},

create: async (req, res, next) => {
  try {
    const created = await prescriptionService.create(req.body,
req.user);
    return sendSuccess(res, 201, 'Prescription created
successfully', created);
  } catch (err) { next(err); }
},

dispense: async (req, res, next) => {
  try {
    const dispensed = await
prescriptionService.dispense(req.params.id, req.user);
    return sendSuccess(res, 200, 'Prescription medicines dispensed
and stock deducted', dispensed);
  } catch (err) { next(err); }
}
};

```

backend/src/controllers/billingController.js (Hospital Invoicing & Payment Processing)

```

import { billingService } from '../services/billingService.js';
import { sendSuccess, sendNotFound } from '../utils/response.js';

export const billingController = {
  getAll: async (req, res, next) => {
    try {
      const bills = await billingService.getAll(req.query);
      return sendSuccess(res, 200, 'Bills retrieved', bills);
    } catch (err) { next(err); }
  },

  getById: async (req, res, next) => {
    try {
      const bill = await billingService.getById(req.params.id);
      if (!bill) return sendNotFound(res, 'Bill not found');
      return sendSuccess(res, 200, 'Bill retrieved', bill);
    } catch (err) { next(err); }
  },

  create: async (req, res, next) => {
    try {
      const created = await billingService.create(req.body, req.user);
      return sendSuccess(res, 201, 'Bill generated successfully',
created);
    } catch (err) { next(err); }
  },

  updateStatus: async (req, res, next) => {
    try {
      const { status } = req.body;
      const updated = await billingService.updateStatus(req.params.id,
status, req.user);
      if (!updated) return sendNotFound(res, 'Bill not found');
      return sendSuccess(res, 200, `Bill status updated to ${status}`,
updated);
    } catch (err) { next(err); }
  }
};

```

backend/database/schema.sql (PostgreSQL Relational Schema & Table Definitions)

```

--
=====
-- CarePoint Hospital Management System (HMS) - PostgreSQL Relational
Schema
--
=====

CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

CREATE TYPE user_role AS ENUM (
  'Administrator', 'Doctor', 'Receptionist', 'Pharmacist', 'Lab
Technician', 'Patient'
);
CREATE TYPE account_status AS ENUM ('Active', 'Inactive',

```

```

'Suspended');
CREATE TYPE appointment_status AS ENUM ('Scheduled', 'Completed',
'Cancelled', 'No Show');
CREATE TYPE prescription_status AS ENUM ('Pending', 'Dispensed',
'Cancelled');
CREATE TYPE medicine_status AS ENUM ('Available', 'Low Stock',
'Expired');
CREATE TYPE lab_test_status AS ENUM ('Requested', 'In Progress',
'Completed', 'Cancelled');
CREATE TYPE bill_status AS ENUM ('Unpaid', 'Paid', 'Cancelled');

```

```

-- 1. Hospital Settings
CREATE TABLE IF NOT EXISTS hospital_settings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  hospital_name VARCHAR(255) NOT NULL DEFAULT 'CarePoint Super
Specialty Hospital',
  email VARCHAR(255) NOT NULL DEFAULT 'contact@carepointhospital.org',
  phone VARCHAR(100) NOT NULL DEFAULT '+1 (800) 456-7890',
  emergency_phone VARCHAR(100) NOT NULL DEFAULT '+1 (800) 911-CARE',
  address TEXT NOT NULL DEFAULT '742 Evergreen Healthcare Ave, Medical
City, MC 54321',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

-- 2. User Profiles
CREATE TABLE IF NOT EXISTS profiles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID UNIQUE,
  full_name VARCHAR(255) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  phone VARCHAR(50),
  gender VARCHAR(20) DEFAULT 'Male',
  role user_role NOT NULL DEFAULT 'Patient',
  status account_status NOT NULL DEFAULT 'Active',
  avatar_url TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

-- 3. Clinical Departments
CREATE TABLE IF NOT EXISTS departments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(150) NOT NULL UNIQUE,
  code VARCHAR(50) NOT NULL UNIQUE,
  head_doctor_name VARCHAR(255),
  total_doctors INTEGER DEFAULT 0,
  status VARCHAR(20) DEFAULT 'Active',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

-- 4. Doctors & Availability
CREATE TABLE IF NOT EXISTS doctors (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  profile_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  department_id UUID REFERENCES departments(id) ON DELETE SET NULL,
  specialization VARCHAR(255) NOT NULL,
  qualification VARCHAR(255) NOT NULL,
  consultation_fee DECIMAL(10, 2) NOT NULL DEFAULT 75.00,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE IF NOT EXISTS doctor_availability (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  doctor_id UUID NOT NULL REFERENCES doctors(id) ON DELETE CASCADE,
  available_days TEXT[] NOT NULL,
  start_time TIME NOT NULL DEFAULT '09:00:00',
  end_time TIME NOT NULL DEFAULT '17:00:00',
  slot_duration_minutes INTEGER NOT NULL DEFAULT 30
);

```

```

-- 5. Patients
CREATE TABLE IF NOT EXISTS patients (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  profile_id UUID REFERENCES profiles(id) ON DELETE SET NULL,
  patient_code VARCHAR(50) NOT NULL UNIQUE,
  date_of_birth DATE NOT NULL,
  blood_group VARCHAR(10) NOT NULL,
  emergency_contact VARCHAR(255),
  allergies TEXT,
  chronic_conditions TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

-- 6. Appointments
CREATE TABLE IF NOT EXISTS appointments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

appointment_code VARCHAR(50) NOT NULL UNIQUE,
patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
doctor_id UUID NOT NULL REFERENCES doctors(id) ON DELETE CASCADE,
department_id UUID REFERENCES departments(id) ON DELETE SET NULL,
appointment_date DATE NOT NULL,
appointment_time TIME NOT NULL,
reason TEXT NOT NULL,
status appointment_status NOT NULL DEFAULT 'Scheduled',
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 7. Electronic Prescriptions & Line Items
CREATE TABLE IF NOT EXISTS prescriptions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    prescription_code VARCHAR(50) NOT NULL UNIQUE,
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    doctor_id UUID NOT NULL REFERENCES doctors(id) ON DELETE CASCADE,
    diagnosis TEXT NOT NULL,
    instructions TEXT,
    status prescription_status NOT NULL DEFAULT 'Pending',
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS prescription_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    prescription_id UUID NOT NULL REFERENCES prescriptions(id) ON DELETE
CASCADE,
    medicine_name VARCHAR(255) NOT NULL,
    dosage VARCHAR(100) NOT NULL,
    frequency VARCHAR(100) NOT NULL,
    duration VARCHAR(100) NOT NULL,
    quantity INTEGER NOT NULL DEFAULT 1
);

-- 8. Pharmacy Medicine Inventory
CREATE TABLE IF NOT EXISTS medicines (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    medicine_code VARCHAR(50) NOT NULL UNIQUE,
    name VARCHAR(255) NOT NULL,
    category VARCHAR(100) NOT NULL,
    stock INTEGER NOT NULL DEFAULT 0,
    min_threshold INTEGER NOT NULL DEFAULT 20,
    price DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
    status medicine_status NOT NULL DEFAULT 'Available',
    expiry_date DATE NOT NULL
);

-- 9. Billing & Invoices
CREATE TABLE IF NOT EXISTS bills (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    invoice_no VARCHAR(50) NOT NULL UNIQUE,
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    total_amount DECIMAL(10, 2) NOT NULL,
    status bill_status NOT NULL DEFAULT 'Unpaid',
    payment_method VARCHAR(50),

```

```

    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

src/index.css (Design System, Medical Color Tokens & Print Utilities)

```

/*
=====
=====
CarePoint Hospital Management System – Design System & Custom Print
Tokens
=====
===== */

@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
    body {
        @apply bg-slate-50 text-slate-900 selection:bg-rose-100
selection:text-rose-900;
    }
}

/* Custom Healthcare UI Scrollbars */
::-webkit-scrollbar {
    width: 6px;
    height: 6px;
}
::-webkit-scrollbar-track {
    background: #f1f5f9;
}
::-webkit-scrollbar-thumb {
    background: #cbd5e1;
    border-radius: 3px;
}
::-webkit-scrollbar-thumb:hover {
    background: #94a3b8;
}

/* Print Styling for Clinical Documents & Invoices */
@media print {
    body * {
        visibility: hidden;
    }
    #printable-area, #printable-area * {
        visibility: visible;
    }
    #printable-area {
        position: absolute;
        left: 0;
        top: 0;
        width: 100%;
    }
}

```